

Colab Training Guide — Full Attention-LSTM (M3)

Run `notebooks/03_model_building.ipynb` on a Google Colab **GPU** to train the attention-LSTM for real (20 epochs + Optuna search) and regenerate the prediction artifacts. The notebook auto-detects the GPU — **you do not edit any epoch/config value by hand.**

The classical models (ARIMA, GJR-GARCH, LightGBM) train in seconds on CPU and their numbers don't change on Colab. Only the LSTM needs the GPU.

★ EASY START (beginner — ~5 minutes, 2 files, 6 steps)

You only need **2 files from your project folder** (`Stock-Market-Trend-Analysis/`):

1. `notebooks/03_model_building.ipynb` ← the notebook
2. `Stock-Market-Colab-Data.zip` ← the data (already made for you, in the project root)

Step 1 — Open Colab. Go to <https://colab.research.google.com> → sign in with Google → **File** → **Upload notebook** → pick `03_model_building.ipynb`. It opens.

Step 2 — Turn on the free GPU. Menu **Runtime** → **Change runtime type** → **Hardware accelerator: T4 GPU** → **Save**. (This is what makes it train for real instead of a 2-epoch test.)

Step 3 — Add ONE setup cell at the very top. Click the notebook's first cell, press **+** **Code** to add a cell *above* everything, and paste this in. It installs the libraries and loads your data:

```
# === SETUP — run this FIRST ===
!pip -q install lightgbm optuna==4.1.0 arch          # tensorflow already on Colab GPU
import os, zipfile
from google.colab import files
os.makedirs("/content/Stock-Market-Trend-Analysis/data/processed", exist_ok=True)
print("Click 'Choose Files' and pick Stock-Market-Colab-Data.zip ...")
up = files.upload()                                # <- a button appears; pick the
zip
z = [k for k in up if k.endswith(".zip")][0]
zipfile.ZipFile(z).extractall("/content/Stock-Market-Trend-
Analysis/data/processed")
print("Ready:", os.listdir("/content/Stock-Market-Trend-Analysis/data/processed"))
```

Step 4 — Run everything. Menu **Runtime** → **Run all**. When the setup cell shows a **"Choose Files"** button, click it and pick `Stock-Market-Colab-Data.zip`. Then it keeps going on its own (~2–5 min on GPU).

Step 5 — Check it trained for real. Near the LSTM cell, the output must say `FULL_TRAIN=True` and `LSTM+Attention (FINAL)` (not "SMOKE"). If it says `False`, your GPU isn't on — redo Step 2 and Run all again.

Step 6 — Download your results. Add a new cell at the **bottom** and run it:

```
import shutil
from google.colab import files
root = "/content/Stock-Market-Trend-Analysis"
shutil.make_archive("/content/results", "zip", root, base_dir="models")
shutil.make_archive("/content/preds", "zip", root, base_dir="data/processed")
files.download("/content/results.zip")          # trained model + tuned params +
metrics                                         #
files.download("/content/preds.zip")           # holdout_predictions.parquet
(LSTM + ensemble filled)
```

Unzip those back into your local project (overwrite `models/` and `data/processed/`) and the LSTM + ensemble holdout numbers are now real. Done.

Common snag: if a cell errors with "file not found", the setup cell (Step 3) didn't run first — click it, run it alone (Shift+Enter), pick the zip, then **Runtime** → **Run all** again.

The rest of this document is the detailed reference (paths, verification, what each artifact is).

0. Set the runtime to GPU

Colab menu → **Runtime** → **Change runtime type** → **Hardware accelerator: T4 GPU** → **Save**. (Free T4 trains this model in ~1–2 min; CPU takes ~15–20 min but still works.)

1. Get the data + notebook onto Colab

The notebook expects everything under `/content/Stock-Market-Trend-Analysis/`. Pick **one**:

Option A — clone from GitHub (if you pushed the repo). First Colab cell:

```
!git clone https://github.com/<you>/Stock-Market-Trend-Analysis.git
/content/Stock-Market-Trend-Analysis
```

Option B — upload the files manually (no GitHub needed). Upload the notebook via **File** → **Upload notebook**, then in the first cell create the folders and upload the 4 required data files:

```
import os
from google.colab import files
os.makedirs("/content/Stock-Market-Trend-Analysis/data/processed", exist_ok=True)
os.makedirs("/content/Stock-Market-Trend-Analysis/data/raw", exist_ok=True)
os.chdir("/content/Stock-Market-Trend-Analysis/data/processed")
files.upload() # pick: train_fe.parquet, val_fe.parquet, holdout_fe.parquet,
feature_roles.json
os.chdir("/content")
```

Required files (from your local `data/processed/`): `train_fe.parquet`, `val_fe.parquet`, `holdout_fe.parquet`, `feature_roles.json`. (The `data/raw/*.csv` snapshots are **not** needed for M3 —

they're only for M1.)

Path handling: you don't set any paths yourself. The notebook's setup cell detects Colab and sets `ROOT = /content/Stock-Market-Trend-Analysis`, then reads `ROOT/data/processed/*.parquet`. As long as the files sit there (Option A or B), there are no path errors. It also `pip installs requirements.txt` if present, else `arch`, `tensorflow`, `lightgbm`.

2. Full 20-epoch GPU training + LSTM Optuna search — automatic

There is **no manual epoch change**. On a GPU runtime `FULL_TRAIN` becomes **True** by itself, which does two things (both gated behind the flag, so local CPU stays a 2-epoch smoke):

1. Runs a **small Optuna search** (15 trials) over an inner purged time-split (last ~250 train dates as inner-val; scaler refit on inner-train only) tuning `units / dropout / learning_rate / batch_size / layers / bidirectional`. Prints `LSTM best config (Optuna): {...}` and saves `models/lstm_best_params.json`.
2. Refits the best config for **20 epochs** with early stopping and fills the holdout `y_pred_lstm`.

Just **Runtime** → **Run all**. Confirm the print near the LSTM cell says `FULL_TRAIN=True`.

The LightGBM **Optuna tuning (\$5b-tune, 60 trials on the purged walk-forward CV)** runs on CPU too, so it executes locally *and* on Colab — its numbers don't need the GPU. Only the LSTM search benefits from it.

(If you're on a CPU runtime but still want the full run, add a cell `import os; os.environ["LSTM_FULL_TRAIN"]="1"` **before** the LSTM cell.)

3. Verify attention + LightGBM + ensemble are running

Check these in the cell outputs, in order:

1. **GPU active** — run once: `import tensorflow as tf; print(tf.config.list_physical_devices("GPU"))` → non-empty.
2. **Attention layer present** — after the LSTM builds, run `lstm.summary()`. You must see a `Softmax`, a `Multiply`, and a `Lambda/reduce-sum` layer (the additive attention pooling over the 60-day window). If those are missing, `attention=True` didn't take.
3. **Full training, not smoke** — the LSTM print shows `FULL_TRAIN=True LSTM+Attention (FINAL)` and ~20 epoch lines (early stopping may end sooner). The model row appears in the \$6 comparison table (smoke never does).
4. **LightGBM ran + used exogenous data** — \$5b prints `exogenous features in LightGBM top-15: [...]` (expect VIX/yield/dollar names) and saves `reports/figures/M3_fig_lgb_importance.png`.
5. **Ensemble used the real LSTM** — \$5c prints `Ensemble GARCH+LSTM (REAL LSTM): {RMSE..., DirAcc...}` (now on the return scale, fully scored — not "illustrative, LSTM=smoke"). The old LGB+LSTM z-blend prints below it as the *rejected* baseline.
6. **Holdout LSTM + ensemble filled** — \$8 prints `Pooled holdout LSTM+Attention: ...` and `Pooled holdout Ensemble GARCH+LSTM: ...`, and `holdout_predictions.parquet` has non-null `y_pred_lstm + y_pred_ens` (first 60/ticker stay NaN by design). The self-audit prints `All N M3 self-audit checks passed` (the count auto-updates). The `[GUARDRAIL]` lines show the tuned-vs-baseline LightGBM PROMOTE/REVERT verdict.

4. Extract + download the artifacts

Everything is written under `ROOT/models/`, `ROOT/data/processed/`, `ROOT/reports/figures/`. Zip and download:

```
import shutil
from google.colab import files
root = "/content/Stock-Market-Trend-Analysis"
shutil.make_archive("/content/m3_trained", "zip", root, base_dir="models")
# models/
shutil.make_archive("/content/m3_preds", "zip", root, base_dir="data/processed")
# incl. holdout_predictions.parquet
shutil.make_archive("/content/m3_figs", "zip", root, base_dir="reports/figures")
# M3 figures
for z in ("m3_trained", "m3_preds", "m3_figs"):
    files.download(f"/content/{z}.zip")
```

What you get (the things that update your M3 report):

File	What it is
<code>models/lstm_attention_final.keras</code>	the trained attention-LSTM weights
<code>models/lstm_best_params.json</code>	the Optuna-selected LSTM config + inner-val RMSE
<code>models/lstm_val_metrics.json</code>	real LSTM val RMSE / MAE / DirAcc / DirAcc_p
<code>models/lgbm_best_params.json</code> , <code>models/lgbm_optuna_study.pkl</code>	tuned LightGBM params + CV summary + the study
<code>models/lgb_tuning_guardrail.json</code>	baseline-vs-tuned holdout RMSE/DirAcc/DM + PROMOTE/REVERT verdict
<code>models/feature_ablation.csv</code>	candidate-feature MI + walk-forward permutation verdicts (KEEP/reject)
<code>models/val_scores.csv</code>	the full val comparison table incl. real LSTM, tuned LightGBM, GARCH+LSTM ensemble rows
<code>data/processed/holdout_predictions.parquet</code>	now has <code>y_pred_lstm</code> + <code>y_pred_ens</code> filled (+ <code>y_pred_arima</code> , <code>y_pred_garch</code> , <code>y_pred_lgb</code> , <code>y_pred_lgb_tuned</code>)
console line <code>Pooled holdout LSTM+Attention: RMSE=... DirAcc=... (binomial p=...)</code>	the real LSTM holdout metrics

Updating the M3 report: replace the placeholder LSTM row in `reports/milestones/M3.md` §6 (val) and §8 (holdout) with the numbers from `lstm_val_metrics.json` and the `Pooled holdout LSTM+Attention` print. RMSE and directional accuracy are already computed and printed — copy them straight in.

Reproducibility note: GPU training has minor run-to-run nondeterminism (seeds pin the graph, not every GPU op). Expect the LSTM numbers to move by a small amount between runs; that is normal and doesn't change the conclusion. A local CPU 20-epoch run of this exact path gives LSTM holdout DirAcc ≈ 0.542 (binomial $p \approx 0.005$) with RMSE slightly worse than naive — a weak, statistically-significant-but-uneconomic directional edge, same pattern as GJR-GARCH. Don't expect the GPU run to beat the naive baseline economically; the EMH ceiling holds.